Git (Version Control):

During previous assignments we have made use of VSCode liveshare, which allowed us to work on the same files at the same time which was really handy for assignments that only required one file. The main issue with liveshare was the fact that one person could only work on the project while the other was working on it as well. This wasn't the biggest issue in the smaller exercises since they could be completed within a single practical session.

When we got to larger assignments with multiple files however, that advantage started becoming less important and the disadvantages started playing a bigger role. Instead of one file, the CBL project required many files with all kinds of purposes. The CBL project also couldn't be completed within a single session, and working outside of working sessions (when it wasn't guaranteed if we both would be available) started becoming necessary.
We used github because of these reasons, as it was easy to learn and integrate into vs code. We both had github accounts, we also downloaded git which gave us git bash where we could run the git commands. After that, creating a repository on github and sharing it and then pushing and pulling from it became easier, as we could see our changes on the website to make sure everything was smooth.

Another advantage of git was the ability to cross reference different versions of our code, if we messed something up we could either revert it or copy code from a working build. The commit history was helpful a lot in these cases. We could see who changed what and the commit message would give info about the work being done.

We learnt a lot of git commands, such as add, pull, push, commit, stash, pop and also learnt to use them in which situations. For example stash was used whenever someone wanted to pull the directory and then "pop" it back when they wanted their stashed changes to be used again. We learnt about branches, merge conflicts and how to correctly commit our changes which the other person could pull. We could have used branches more to experiment with them and to learn more about them, but since our tasks were quite divided we didn't really feel the need to use them.

We believe it was challenging and a little annoying to first get used to git and its commands and how it worked, but after spending time on it we figured out how to use it and make it better for us. Github is also used in the professional field, so it is good that we have some experience related to it, and learning it is definitely going to benefit us. It had a great positive impact on our project since without it we wouldn't have been able to collaborate without running into problems.

Maven:

Before we started using maven we used to dump all our projects into one combined map which made it very chaotic, especially when the assignments started to require more than one file. Maven gave a structure to build larger projects from. With this project we didn't even use the full potential which only shows the potential order it could give in bigger and more complicated projects.

We created a maven project using the maven quickstart template, which gave us the files and the structure. It was easy to understand that the src folder contained our files that we work on, and the test folder is for testing when we need to. It gave us the pom.xml file containing our name of the project which was poker and our group which is cblproject that we named. The pom.xml was the file that was very important to the maven. It showed the java version being used for compiling which is shown in maven.complier.source which we set to 17.

project.build.sourceEncoding was set to utf-8, since we used the UTF for the symbols for the suits, which was important as this will make sure the UTF encoding works on all devices. This was one of the tags we learnt about maven, and we implemented in our project which was essential for the characters of the suits to show up.

We also added target/ to git ignore. We learnt that from our research that Maven puts everything it generates, like compiled classes and the packaged jar, in a target folder. This folder is rebuilt from the source on every build, so we added /target to our .gitignore. That way only our source code and pom.xml get committed, and the repository doesn't fill up with files that change on every build and would only cause pointless merge conflicts.
It also shows how we connected the git and maven together and used them in our project.

We did leave some of the tags in the pom.xml just how they were when the project was created, for example there as a tag to show your website, but we don't really have a website of our own so we left that just how it was. We also had a Junit extension that we added, but we didn't really feel the need to do any much testing, so we could have done that as well. Overall we didn't utilise the full capabilities of maven, but we did learn some
The impact of this on the project was that understanding the pom.xml would scale in a more complex project and how editing it would be done when installing new plugin and extensions and dependencies.